

The Modern Computer Vision Pipeline

A Technical Research Report

Ahsan Rehman | The University of Faisalabad | Computer Vision Fellowship 2026, Visibility Bots Innovation Lab

1. Introduction

Computer Vision (CV) is the field of artificial intelligence that enables machines to interpret and understand visual information from the world, mimicking the way human vision extracts meaning from images and video. Modern CV systems are built as a pipeline: a sequence of well-defined stages that transform raw pixel data into actionable understanding. Each stage builds on the output of the previous one, moving progressively from raw sensor data to high-level semantic decisions. This report explains the seven core stages of a modern computer vision pipeline: Image Acquisition, Image Processing, Object Detection, Classification, Segmentation, Tracking, and Deployment.

2. Image Acquisition

Image acquisition is the entry point of the pipeline, where visual data is captured from the physical world using sensors such as digital cameras, webcams, LiDAR, or specialized industrial imaging equipment. The quality of acquisition directly affects every downstream stage: factors like resolution, frame rate, lighting conditions, sensor noise, and lens distortion all influence how usable the raw data is. Acquisition may be a single static image, a continuous video stream, or a stereo/depth pair for 3D vision tasks.

Real-world example: Autonomous vehicles use an array of cameras, LiDAR, and radar sensors mounted around the vehicle to continuously acquire a 360-degree view of the surrounding environment.



3. Image Processing (Preprocessing)

Once an image is acquired, it is rarely ready for direct analysis. Preprocessing cleans, standardizes, and enhances the raw image so that later stages can extract reliable features. Common operations include resizing and normalization, grayscale conversion, noise removal (e.g., Gaussian blur), thresholding, histogram equalization, and edge detection (e.g., the Canny algorithm). Morphological operations such as erosion and dilation are also used to refine shapes and remove small artifacts before feature extraction.

Real-world example: Medical imaging systems apply denoising and contrast enhancement to X-ray or MRI scans before a diagnostic algorithm attempts to identify abnormalities.

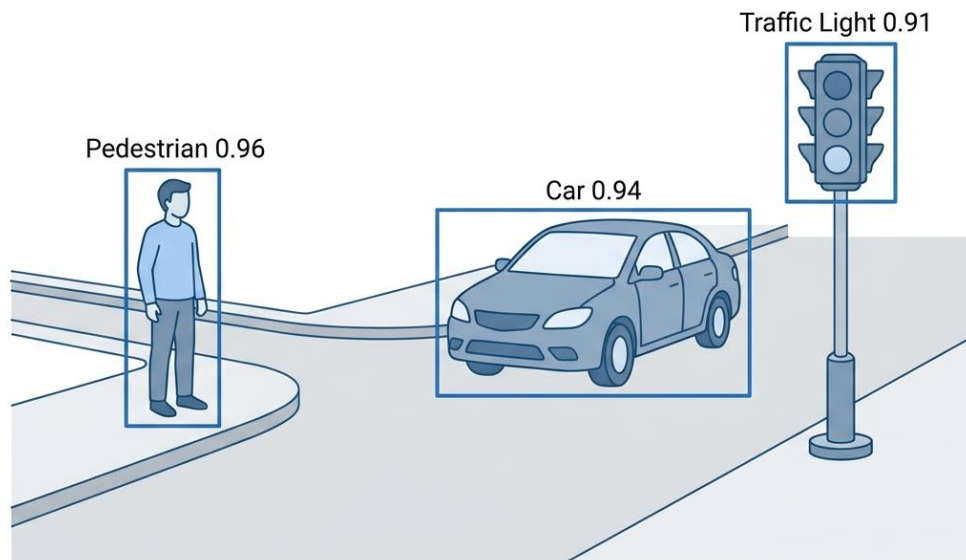
4. Object Detection

Object detection identifies what objects are present in an image and where they are located, typically by drawing bounding boxes around each instance. Unlike simple classification, detection must handle multiple objects of

different classes within a single frame. Classical approaches relied on handcrafted features (e.g., Haar cascades, HOG), while modern systems use deep learning architectures such as YOLO (You Only Look Once), SSD, and Faster R-CNN, which perform detection in a single or two-stage forward pass through a convolutional neural network.

Real-world example: YOLO-based systems are used in retail smart-checkout stations to detect and localize multiple products in real time as they pass a camera.

Object Detection Output



5. Classification

Classification assigns a label to an image or a detected region, answering the question 'what is this?' rather than 'where is it?'. It is often applied after detection to categorize each identified object, or on its own for whole-image labeling tasks. Convolutional Neural Networks (CNNs) such as ResNet, VGG, and EfficientNet are the dominant architecture family for classification, learning hierarchical visual features directly from data rather than relying on manual feature engineering.

Real-world example: Agricultural drones classify captured leaf images as healthy or diseased to help farmers target treatment to specific crop regions.

6. Segmentation

Segmentation goes a step further than detection by assigning a label to every individual pixel in an image, producing a precise outline of objects rather than a coarse bounding box. Semantic segmentation labels each pixel by class (e.g., 'road', 'car', 'pedestrian') without distinguishing between instances, while instance segmentation (e.g., Mask R-CNN) separately identifies each object instance. Segmentation is essential when exact shape and boundary information matters.

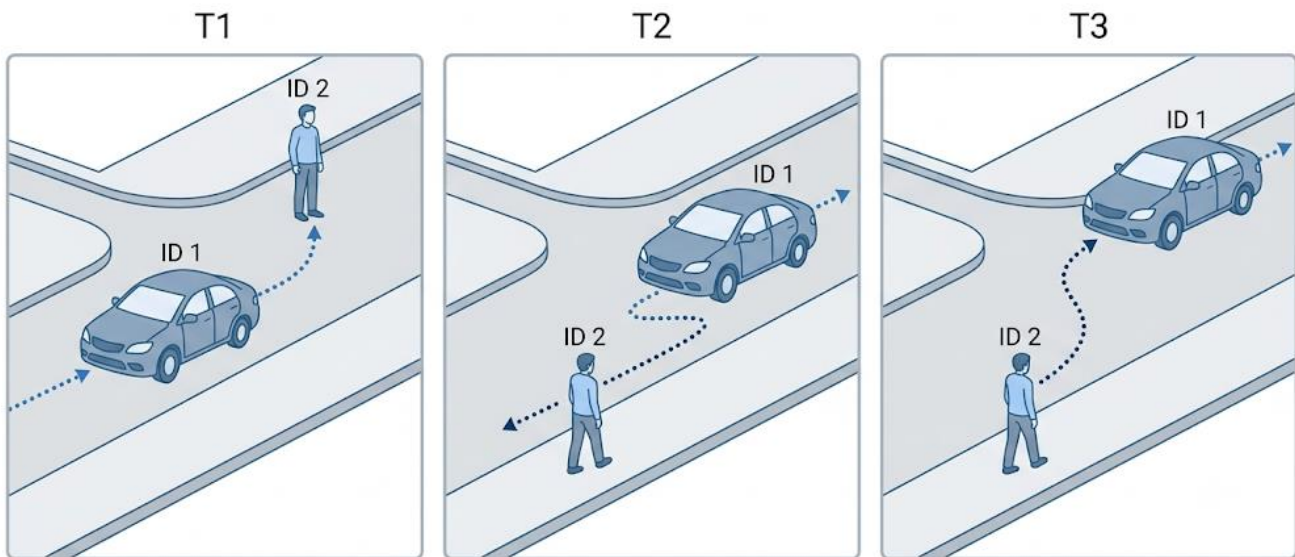
Real-world example: Self-driving cars use semantic segmentation to distinguish the drivable road surface from sidewalks, lane markings, and obstacles at the pixel level.

7. Tracking

Tracking maintains the identity of detected objects across consecutive video frames over time, rather than treating each frame independently. This allows a system to understand motion, trajectory, and behavior. Common approaches include the Kalman Filter for motion prediction, and algorithms like DeepSORT and ByteTrack, which combine detection with appearance-based re-identification to keep track of objects even through brief occlusions.

Real-world example: Sports analytics platforms track individual players and the ball across a match broadcast to generate heatmaps, speed statistics, and tactical insights.

Multi-Object Tracking Sequence



8. Deployment

Deployment is the final stage, where a trained and validated CV model is integrated into a real-world application or system. This involves optimizing the model for the target hardware (e.g., using quantization or pruning for edge devices), building an inference pipeline, and exposing the system via an API, embedded application, or edge device. Deployment environments range from cloud servers to resource-constrained edge devices such as Raspberry Pi, NVIDIA Jetson, or mobile phones, each requiring different trade-offs between accuracy, latency, and power consumption.

Real-world example: Manufacturing quality-control systems deploy lightweight CV models on edge devices directly on the factory floor to detect product defects in real time without relying on cloud connectivity.

9. Conclusion

The modern computer vision pipeline is a structured progression from raw pixels to real-world decisions. Each stage — acquisition, processing, detection, classification, segmentation, tracking, and deployment — addresses a distinct sub-problem, and modern systems increasingly rely on deep learning to power the middle stages while classical image processing techniques remain essential for preprocessing and efficiency. Understanding this pipeline as a whole, rather than any single stage in isolation, is fundamental to building robust, production-ready computer vision systems.

References

1. Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). Springer.
2. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.
3. He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. ICCV.
4. OpenCV Documentation. (2026). Open Source Computer Vision Library. <https://docs.opencv.org>
5. Ultralytics. (2026). YOLOv8/YOLOv11 Documentation. <https://docs.ultralytics.com>
6. Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple Online and Realtime Tracking. ICIP.